Langage Swift pour application mobile

Par F. Guerouate

Swift

« Est un langage de programmation moderne qui est sûr, rapide et interactif »

Apple

Annoncé le 2 juin 2014, lors de la Worldwide Developer Conferance (WWDC), implementé par Chris Lattner

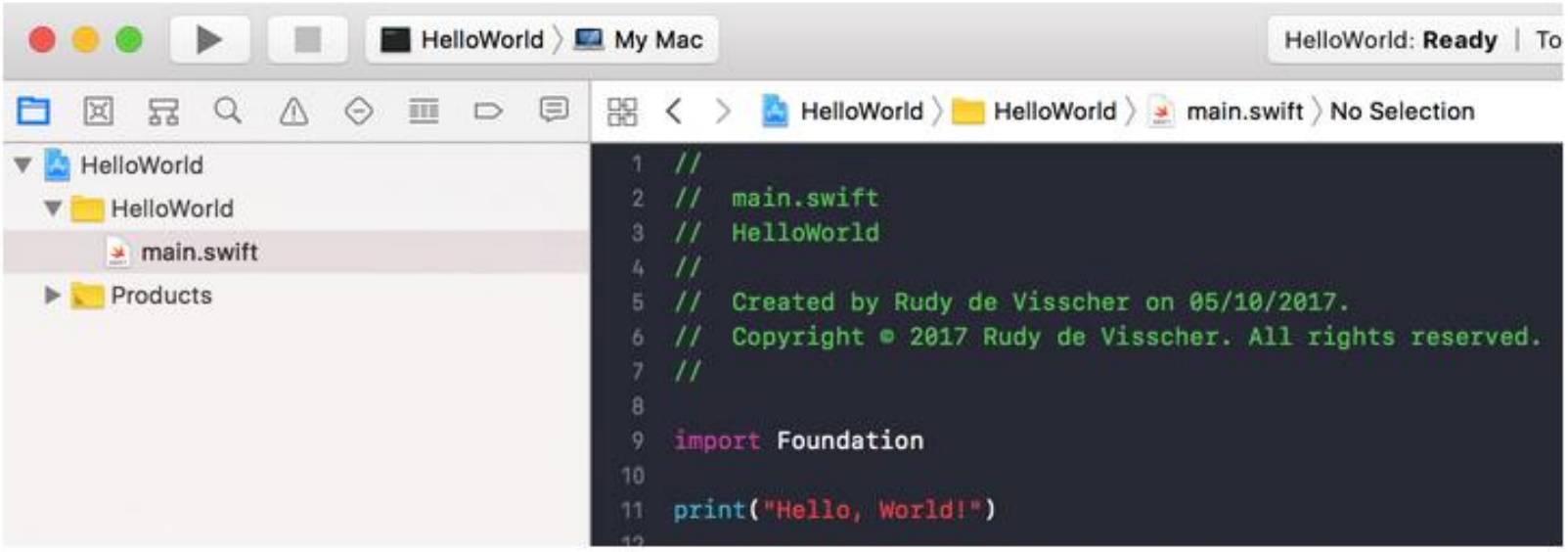
Permet de créer des applications sous les systèmes iOS, OS X et WatchOS

L'IDE

Un seul choix: Xcode

Un seul système: OS X

- Xcode est un environnement de développement pour macOS, watchOS, tvOS et iOS. Il est connu pour sa simplicité et surtout pour ses astuces proposées pour développer plus rapidement.



main.swift

Programme

- 1- les bases du langage Swift
- 2- la POO en Swift
- 3- Utilisation de l’interface graphique iOS
- 4- utilisation de l’interface graphique OS X

Les variables

- var nombreDEtudiant=25
- var nombreDEtudiant: Int
- var nombreDEtudiant: Int = 20

- var nombreDEtudiant: Int
- var pi: Double
- var phrase: String
- var information: Bool

- Si vous déclarez une variable, les prochaines affectations à cette même variable doit rester du même type. Par exemple, si je déclare une variable de type entier. Je ne pourrais modifier la variable que par des nombres entiers à l'avenir.

Les constantes

- Pour déclarer une constante en Swift :
  - let pi = 3.1415

Les tuples

- un tuple peut contenir plusieurs valeurs de n'importe quel type. Une variable ou une constante peut être un tuple.
  - let tuple = (valeur1, valeur2, ...)
  - var tuple = (valeur1, valeur2, ...)

- let JoursSemaine= (« lundi», « Mardi », ...)
- var tuple = (2.5, « bonjour »,2, ...)

La concaténation

- let nombre1 = 1
- let nombre2 = 2
- let nombre = String(nombre1) + String(nombre2)
- print(nombre)

Les conditions

```
let age: Int = 32

if age >= 18 {
    print("Vous êtes majeur.")
} else {
    print("Vous êtes mineur.")
}
```

Switch case

```
let note = 12

switch note {
case 0..<10:
    print("Vous n'avez pas la moyenne, vous n'avez donc pas de mention.")

case 10..<12:
    print("Vous avez la moyenne, mais vous n'obtenez pas de mentions.")

case 12..<14:
    print("Vous avez obtenu la mention assez bien.")

case 14..<16:
    print("Vous avez obtenu la mention bien.")

case 16..<18:
    print("Vous avez obtenu la mention très bien.")

case 18...20: // Ici on inclut évidemment le 20 !
    print("Vous avez les félicitations du jury !")

default:
    print("Navré, il faut avoir une de ces notes pour avoir une mention.")
}
```

Les boucles

```
var nbDeLignes: Int
for nbDeLignes in 1...1000 {
    print("\(nbDeLignes). Bonjour.")
}
```

```
for _ in 1..<1001 {

  print(  "Bonjour.")

}
```

Les boucles

```
for var i = 0; i < 10; i++ {
}

for i in 0...10 {
}

var array = ["item 1", "item 2"]
for item in array {
}
```

Les tableaux et les dictionnaires

Les tableaux

- Déclaration d'un tableau en swift
  - let tableau = [valeur1, valeur2, ..., valeurN]
  - var tableauEntiers: [Int] // il faut pas oublier l'initialization du tableau
  - Var tableauNoms:[String]
  - Var T=[Int]()

Exemple:

```
let prenoms = ["Sara", "Ali", "Mohammed", "Maha"]
print(prenoms[2])
```

```
var prenoms = [”Sara", ”Ali", ”Mohammed", ”Maha”]
// On ajoute en fin de tableau
prenoms = prenoms + [ ’ Nada"]
print(prenoms)
// Ou bien encore en début du tableau
prenoms = ["Jean"] + prenoms
print(prenoms)
// pour concaténer plus rapidemment en fin de tableau
prenoms += [ ’Nada"]
```

- Parcourir le tableau

```
let Tabprenoms=["sara","Maha","mohammed","Ali"]
var i:Int8
for i in 0..<Tabprenoms.count {
print("- "+Tabprenoms[i]+" -")
}
```

```
for A in Tabprenoms{
print(A)
}
```

- Quelques fonctions qui gèrent les tableaux:

```
var name=["sara","toto","pim"]
name.insert("lala", at: 1)
print(name)
name.remove(at: 0)
print(name)
name.index(of: "sara") (nil  s'il n exist pas)
```

```
for i in 0..<name.count
{
        print("\(i):\(name[i])")
}
for (i ,ch) in name.enumerated()
{
         print("\(i):\(ch)")
}
```

Les dictionnaires

clé

Valeur

```
var dictionnaire: [type1: type2]
// type1 peut être identique à type2

// Par exemple
var dictionnaire: [String: Int]
let dictionnaire :Dictionary<Int, String> = [:]
```

Dans le dictionnaire:

- les valeurs ne se rangeront plus dans un ordre spécifique (de 0 à n pour les indices du tableau).

- Une clé est unique.

- Parcourir le dictionnaire

```
let dic=["Nom":"xxxx","Prenom":"sara","Adresse":"yyy
45 yyyy","Ville":"Rabat"]
for (cle,valeur) in dic{
      print(cle + " : ” + valeur)
}
```

- Déclarer un dictionnaire vide

```
var monDictionnaireVide = [String:Int]()
Ou
var monDictionnaireVide: [String: Int] = [:]
```

- Ajouter et supprimer des éléments

```
var countries = ["FR": "France", "IT": "Italie", "UK": "United King"]
// Je modifie la valeur, car la clé "UK" existe déjà
countries["UK"] = "United Kingdom"
// J'ajoute une valeur, car la clé "ES" n'existe pas
countries["ES"] = "Espagne"
```

```
countries.removeValue(forKey: "ES")
// L'association "ES": "Espagne" est supprimée
```

```
Var D= [ "France": "Paris", "Chine":"Pékin",   "
Maroc":  " Rabat"]
let dictKeys = Array(D.keys)

print(dictKeys)

["Chine",  "Maroc", "France"]
```

Les fonctions en Swift

```
func Nomfonction(parametre1: Type, parametre2: Type, ...)
{  // Instructions
 }
```

```
func nomDeLaFonction(parametre1: Type, parametre2: Type, ...) -> TypeRetour
{
    // Instructions

    return laValeurARetourner
}
```

- Lorsqu'on appel une fonction qui a plusieurs paramètres, on doit préciser le nom du paramètre utilisé lors de la déclaration, à partir du deuxième.

Nomfonction(valeur1, paramètre2: valeur2, ...)

Les fonctions en Swift

Pour Swift on peut attribuer 2 mots à un paramètre:

Exemple:

```
func disBonjour(A prenom: String) {
    print("Bonjour " + prenom + " !")
}
```

Pour appeler la fonction on l'appel avec le premier nom du paramètre:
disBonjour(A: "Sara")

On peut aussi omettre le nom du paramètre lorsqu'on appelle la fonction

```
func disBonjour(_ prenom: String) {
        print("Bonjour " + prenom + " !")
        }
disBonjour( "Sara ")
```

- <u>Fonction passer comme paramètre ou type de retourne</u>
  - Pour indiquer une fonction en tant que paramètre d'une fonction, il faut mettre sa déclaration dans les parenthèses de la fonction.

func nom_fonction(fonct : (type para .)->type retour)->type retour

Pour appeler la fonction:

No_fonction(nom_fonction_paramètre)

Fonction de saisi

- // Fonction permettant de demander à l'utilisateur d'entrer un nombre et de le convertir en entier

```
func input() -> Int {
    let strData = readLine();
    return Int(strData!)!
}

var nombreEntre: Int
nombreEntre = input()
```

- print("Please enter number 1")
- var num1 = Int(readLine()!)! print("Please enter number 2")
- var num2 = Int(readLine()!)!
- var sum = num1 + num2 print("The sum of \(num1) and \(num2) is \(sum)")

Les fermetures

- Une fermetures (closure) est un bloc d’instruction qui n’est pas nommé.
- C’est une fonction sans nom, qui possède des paramètres et une valeur de retourne
- L’avantage est que ce bloc peut passé en paramètre d’une fonction, c’est une seconde alternative au passage d’une fonction, en tant que paramètre, en Swift, ou vous pouvez la stocker comme une propriété d'un objet.

- **La flexibilité**

Vous avez déjà appris que les fonctions peuvent être incroyablement puissantes et flexibles. Parce que les fonctions sont des fermetures, les fermetures sont aussi flexibles.

Cependant, toutes les fermetures ne sont pas une fonction

- En Swift , une fermeture ce déclare comme ceci:

```
{ (paramètres) -> Type Retour in
    // Vos instructions + return
}
```

- Une fermeture commence et se termine par un accolade, enroulant les paramètres, le type de retour et le corps de fermeture.
- Lorsqu’on utilise une fonction qui prend en paramètre une autre fonction. On peut directement mettre tous le bloc ci-dessous dans les paramètres.

- Exemple:

```
 // Déclarez la variable myVar1, avec le type de données, et assignez la valeur.
var myVar1 : () -> () =  {

    print("Hello from Closure 1");
}

// Déclarez la variable myVar2, avec les types de données, et assignez une valeur.
var myVar2 : () -> (String)  = { () -> (String) in

    return "Hello from Closure 2"

}

// Déclarez la variable myVar3, avec les types de données, et assignez la valeur.
var myVar3 : (Int, Int) -> (Int)  =  { (a : Int, b: Int) -> (Int) in

    var c : Int =  a + b

    return c
}
```

```
func test_closure()     {

    // Exécutez de la fermeture (Closure).
    myVar1()

    // Exécutez de la Closure et obtenir la valeur de retour.
    var str2 = myVar2()

    print(str2)

    // Exécutez de la Closure, passez (pass) les paramètres
    // et obtenez une valeur de retour.
    var c: Int =  myVar3(11, 22)

    print(c)

}
```

- Fonction avec fermeture (closure)

Fonction est un cas particulière de Closure. Fonction est une Closure nommé on peut dire que la Closure est une Fonction anonyme.

- **Exemple :**

```
func sum(a: Int, b: Int) -> Int {
  return a + b
}
```

Function

```
{ (a: Int, b: Int) -> Int in
  return a + b
}
```

Closure

```
func sum(a: Int, b: Int) -> Int {
  return a + b
}

var value = sum(1, 2)
```

```
var myClosure = { (a: Int, b: Int) -> Int in
                  return a + b
                }

var value = myClosure(1, 2)
```

- **La fermeture anonyme**

Quand on déclare une Fermeture, c'est pas nécessaire d'écrire le nom des paramètres. Ces paramètres peuvent être références par $0, $1,...

Exemple 1:

```
1   import Foundation
2
3   // Déclarez une Closure de la manière habituelle.
4   var myClosure : (String, String) -> String
5
6       = { (firstName: String, lastName: String) -> String in
7
8
9           return firstName + " " + lastName
10  }
11
12  // Déclarez une Closure de manière anonyme (anonymous).
13  // (Ignorez les noms de paramètres).
14  var anonymousClosure : (String, String) -> String
15
16      = {
17          // En utilisant
18          // $0: Pour le premier paramètre
19          // $1: Pour le deuxième paramètre.
20          return $0 + " " + $1
21  }
```

- Note: $0, $1,... sont des paramètres anonymes. Ils sont uniquement utilisés dans la Fermeture Anonyme. Si vous utilisez la Fermeture normal, vous recevrez un message d'erreur:

- Les arguments de fermeture anonymes ne peuvent pas être utilisés dans une fermeture comportant des arguments explicites

- **Exemple 2:**

```
import Foundation



func test_anonymousClosure()  {

    // Déclarez une variable du type de Closure.
    var mySum : ( Int, Int ) -> (Int)

    // Assignez une Closure annonyme pour lui.
    // $0: Pour le premier paramètre
    // $1: Pour le deuxième paramètre.
    mySum =  {
        return $0 + $1
    }



    var value = mySum(1, 2)

    print(value)

}
```

Les énumérations

- Une énumération permet de définir un type commun pour une liste de valeurs personnalisées.
- De la même manière que la déclaration switch, enum's dans Swift peut à première vue ressembler à une variante légèrement améliorée de la déclaration bien connue enum
- les énumérations de Swift lui permettent d'être utilisées dans un éventail de scénarios pratiques beaucoup plus large que les énumérations ordinaires. En particulier, ce sont d'excellents outils pour manifester clairement les intentions de votre code.

- "Les énumérations déclarent des types avec des ensembles finis d'états possibles et de valeurs correspondantes. Avec l'imbrication, les méthodes, les valeurs associées et la correspondance de modèle, les énumérations peuvent définir des données organisées de manière hiérarchique."

- Définir les énumérations de base

```
enum NomEnumeration{
        case valeur1
        case valeur2
        …
                }

enum NomEnumeration{
        case valeur1, valeur2, …
        }
```

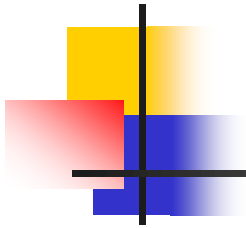
- Pour créer des variable de type énumération:

  * Var V=NomEnumeration.valeur

  * V=.valeur

On peut l'utiliser dans switch case:

```
switch V{
        case.valeur1:action1
        case.valeur2:action2
        …
  }
```



Exemple:

- Voir exemple1 énumération

```
enum Mouvement{
        case gauche
        case droit
        case haut
        case bas
        }
```

<u>Exemple</u>:

- Voir exemple1 énumération

```
enum IOSType{
        case iPhone
        case iPad
        case iWatch
                }
Var myDevice=IOSType.iPhone
```

- Si le type d'une énumération est connu ou peut être déduit, vous pouvez utiliser la syntaxe à points pour les membres.

```
// dans ce cas le type est connu
var myDevice: iOSType = .iPhone

// dans ce cas le type peut être déduit
if myDevice == .iPhone { print(”j’ai un iPhone!") }
```

- Les énumérations Swift peuvent stocker des valeurs associées de tout type et le type de valeur peut être différent pour chaque membre. Par exemple, vous souhaiterez peut-être stocker un modèle d'appareil pour iPhone et iPad (par exemple, "mini" pour l'iPad ou "6 Plus" pour l'iPhone).

```
enum IOSType{
        case iPhone
        case iPad
        case iWatch
            }
```

Les valeurs associées

<u>Exemple</u>:

```
enum IOSType{
        case iPhone(String)
        case iPad(String)
        case iWatch
                }

var myDevice = iOSType.iPhone("6")
```

- Vous pouvez obtenir les valeurs associées en utilisant une instruction switch:

```
var myDevice = iOSType.iPhone("6")

switch myDevice {
        case .iPhone(let model): print("iPhone \(model)")
        case .iPad(let model): print("iPad \(model)")
        case .iWatch: print("iWatch")
        default: print("not an iOS device")
}
```

Les valeurs associées

- Les valeurs associées sont un moyen fantastique d'attacher des informations supplémentaires à une énumération.

```
enum codes {
//code-bar
    case UPCA(Int,Int,Int,Int)
    case QRcode(String) }

Var code=codes.UPCA(3,98987,1,0)
    code=.Qrcode(«  http//…..»)
```

Valeurs Enum

- On peut affecter des valeurs aux cas des énumérations: valeurs brutes:
  - Il faut définir un type natif unique pour tous l' énumération
  - Chaque cas doit avoir une seule et unique valeur
  - La valeur sera de même type pour tous les cas

  (voir exemple 2)

Programmation orienté Objet pour Swift

- Class exemple{

//attributs

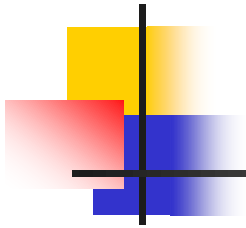
Func

}

- Exemple et difference entre Java et Swift.
  - Voir exemple Livre

//le constructeur

```
init(auteur:String,titre:String,editeur:String,annee:Int,nbrpage:Int) {
        self.auteur=auteur
        self.titre=titre
        self.editeur=editeur
        self.annee=annee
        self.nbrpage=nbrpage
        }
```



Pour déclarer un objet livre:

```
let Livre1 = Livre(auteur:"taher benjelloun",titre:"les enfants vide",editeur:"fff",annee:2018,nbrpage:200)
```

Les structures

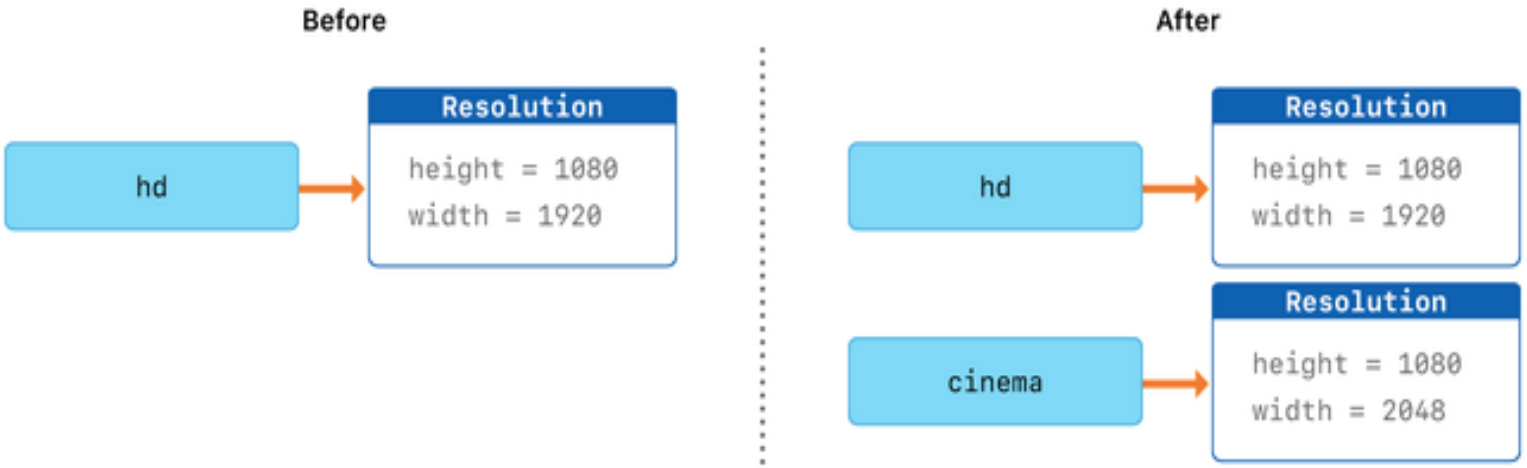
- Swift propose les structures avec quelques différences par rapport aux classes:
  - Le mot clé est struct
  - Il n y a pas d'héritage avec les structures
  - Les classes sont des types par *référence* et les structures des types par *valeur*.

- Une structure possède un constructeur par défaut ➔
  - On peut initialiser les attributs membres directement dans les parenthèses de la déclaration si on veut.
  - let S=nom_struct(paramt1:valeur,paramt2:valeur, …)

```
struct Resolution{
        var largeur : Double=10.5
        var hauteur : Double=5.2
}
// let res=Resolution()
//ou
//let hd=Resolution(largeur: 1080,hauteur: 1920)

//?type par valeur
var cinema=hd

cinema.largeur=2048
```


Before
hd
Resolution
height = 1080
width = 1920
After
hd
Resolution
height = 1080
width = 1920
cinema
Resolution
height = 1080
width = 2048

Classes où structures

- <u>Exemple structure:</u>

Créer un struct Personne avec les propriétés : nom, age, sexe, niveauScolaire

Créer quelques personnes les mettre dans un tableau

Ecrire une fonction qui prend en paramètre un tableau des personnes et qui renvoie un tableau composé des personnes de même niveau

Héritage et polymorphisme

Héritage

```
Class class_fille : classe_mere{
        //attributs et méthodes hérité
        //propre attributs et méthodes
}
```

L héritage multiple est interdit en Swift

Si la classe mère a un constructeur la class fille le possèdera après l'héritage

- Le Swift permet l'encapsulation par des marqueurs de visibilités (public, private et internal par défaut).
  - **private** : visibilité de l'entité seulement depuis le fichier
  - **internal** : visibilité de l'entité depuis d'autre fichiers sources du projet
  - **public** : visibilité de l'entité depuis d'autre fichiers sources du projet et depuis les imports (plus haut niveau)

- <u>Le polymorphisme</u>

Si on veut qu'une méthode héritée ait une fonctionnalité propre a la classe fille

C'est un concept permettant de fournir une interface unique à des entités pouvant avoir différents types.

- Les méthodes publiques seront hérités et peuvent être réimplémenté par la notation « **override »** (overridding – polymorphisme en anglais).

```
Override func nom_fonct{

                //nouvelles instructions
}
```

- Si on conserve le corps de la méthode de base et on ajoute d'autres instructions:

```
Override func nom_fonct{

    super.Nom_methode
    //nouvelles instructions
}
```

Les protocoles

- Les protocoles (⇔ interface en Java) est un modèle que va suivre une classe.

- Certain objets peuvent avoir des fonctionnalités commune

- Il est possible de spécifier des méthodes ainsi que des champs. L'ensemble du corps du protocol devra être réimplémenté par la nouvelle classe.

- Exemple: les objets carré, rectangle et triangle.
- Ces objets peuvent avoir des fonctionnalités commune : calculer Air et périmètre

- Déclarer un protocole:
  - Ecrire uniquement l'entête des fonctions

```
func NomMethode(arg1:type,arg2:type …)
```

Le corps des méthodes sera définie dans les classes qui seront associé a ce protocole

- Créer un protocole en swift:

```
protocol NomProtocole{

        // entête des methodes qui seront définies dans les
classe associé au protocole

                }
```

- Utiliser un protocole:

```
class NomClass:NomProtcole{
        // définition des méthodes dans l'entête est présent dans le protocole
        // les méthodes propre à la classe

}
```

- Associé une classe à plusieurs protocoles:

```
class NomClass:NomProtcole1, NomProtocole2, … {
        // définition des méthodes dans l'entête est présent dans le protocole
        // les méthodes propre à la classe

}
```

- Associé une classe à plusieurs protocoles et a une classe mère:

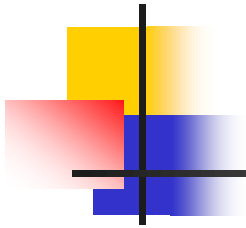
```
class NomClass:NomClasseMere,NomProtcole1, NomProtocole2, … {
        // définition des méthodes dont l'entête est présent dans le protocole
        // les méthodes propre à la classe

                                }
```

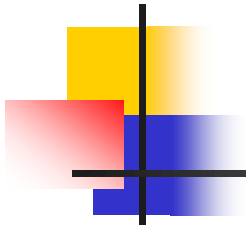
- Comparer deux objets s’ils sont de même instance (===) où (!==)

Les variables optionnelle

- Une optionnelle est une variable qui peut être affectée où non à une valeur. Une variable optionnelle se voit attribuée une valeur « nil » par défaut grâce au « ? ». Au contraire si notre optionnelle doit avoir une valeur sûr on lui attribuée « ! »



- var jeSuisOptionnel: Int? // Ceci est un optionnel
- jeSuisOptionnel // Le Playground affiche nil
- var jeNeSuisPasOptionnel: Int // Ceci est un entier
- jeNeSuisPasOptionnel // Le Playground plante…



Les optionnelles

- Les variables de type optionnel ont:
  - Soit une valeur
  - Soit pas de valeur -> nil
  - Voir exemple

Déclarer une variable Les optionnelle

- Var opt:Int? = 7
  - Opt = nil
  - Opt = 22
    - Une chose qui est impossible pour les autres variables, pas de changement de type de valeur